CSCI 3104 Fall 2024
Instructor: Dr. Lijun Chen

Problem Set 3

Due Date ..... September 24, 2024
Name ..... **Hyma Jujjuru**
Student ID **110743269**
Collaborators ..... **List Your Collaborators Here**

Contents

Instructions

- The solutions **should be typed**, using proper mathematical notation. We cannot accept hand-written solutions. Here's a short intro to LaTeX.
- You should submit your work through the **class Gradescope page** only (linked from Canvas). Please submit one PDF file, compiled using this LaTeX template.
- You may not need a full page for your solutions; pagebreaks are there to help Gradescope automatically find where each problem is. Even if you do not attempt every problem, please submit this document with no fewer pages than the blank template (or Gradescope has issues with it).
- You are welcome and encouraged to collaborate with your classmates, as well as consult outside resources. You must **cite your sources in this document. Copying from any source is an Honor Code violation. Furthermore, all submissions must be in your own words and reflect your understanding of the material.** If there is any confusion about this policy, it is your responsibility to clarify before the due date.

- Posting to **any** service including, but not limited to Chegg, Reddit, StackExchange, etc., for help on an assignment is a violation of the Honor Code.
- You **must** virtually sign the Honor Code (see Section). Failure to do so will result in your assignment not being graded.

Honor Code (Make Sure to Virtually Sign) [10 pts]

Problem HC.
- My submission is in my own words and reflects my understanding of the material.
- Any collaborations and external sources have been clearly cited in this document.
- I have not posted to external services including, but not limited to Chegg, Reddit, StackExchange, etc.
- I have neither copied nor provided others solutions they can copy.

Agreed (signature here). I agree to the above, Hyma Jujjuru. □

1 Greedy Algorithms

1.1 Problem 1 [30 pts]

Problem 1. Suppose you fly a drone from Boulder to Denver to deliver a package. The battery of the drone, when fully charged, has enough energy to fly k kilometers. Along the flight route from Boulder to Denver, there are n wireless charging stations where the drone can stop to charge its battery. Denote by $l_1 < l_2 < \cdots < l_n$ the locations of the charging stations along the route, with l_i being the distance from Boulder to the i-th charging station. The distance between neighboring charging stations is assumed to be at most k kilometers.

Your objective is to make as few charging stops as possible along the way.

Tasks

1. Describe the greedy choice property and optimal substructure property for this problem. Give an optimal greedy algorithm to determine at which charging stations the drone should stop to minimize the number of charging stops.

2. State the associated time complexity of your algorithm.

3. Prove that your algorithm gives an optimal solution (Proof of correctness).

Rubric	**Points**
Correct greedy algorithm and pseudocode	14 Points
Analysis of pseudocode	6 Point
Proper use of mathematical notation	2 Point
Proof of correctness using exchange arguments	8 Points
Total	**30 Points**

Proof.

1. **Greedy Choice Property**: We greedily choose the next charging station that maximizes the distance between charging stops. That is the drone can fly k kilometers on a full charge so we choose the charging station that is as close to k kilometers away from the current location as possible without going over to maximize the distance traveled and minimize the number of charging stops.

   Optimal Substructure Property: If we can find a solution that provides the optimal number of stops for 15 km, then we can use the same strategy for the whole journey to produce an optimal solution.

   Optimal Greedy Algorithm:
   The optimal greedy algorithm needs to produce the fewest number of stops over the distance between Boulder and Denver. To do this, we find the charging station that is less than or equal to k kilometers from the current position which is initially 0 kilometers from Boulder. We repeat this until we reach Denver.
   There are n charging stations between Boulder and Denver, so the list containing the locations of all the charging stations between Boulder and Denver (L) has a length of n.

```
1    fewestChargingStops(L, k)
    //L stores the locations of charging stations
2        currPos = 0
3        stops = []
4        while currPos < dist_To_Den
5            nextStation = -1
6            for i = currPos to L.length - 1
7                if L[i] <= currPos + k
8                    nextStation = L[i]
9                else
```

```
10                      break
11              if nextStation = -1
12                  return "No possible route found."
13              stops.append(nextStation)
14              currPos = nextStation
15          return stops
```

2. **Time complexity**:
   The time complexity of the algorithm above is $O(n^2)$ because the while loop can run for a maximum of n times but the for loop inside can also run up to n times, so the overall time complexity would be n^2.

3. **Proof of Correctness**:
   Assume the greedy solution G has the stops $(l_2, l_4, l_5, ...l_n)$ and the optimal solution O has the stops $l_1, l_3, l_4, ..., l_n$. Assume the greedy solution chooses stops that are farther such as $l_2 > l_1$ and $l_4 > l_3$.

   The optimal solution, O, still reaches the destination in 4 stops in some charging stops. However, it does not go farther than the greedy solution, G. The drone could hypothetically swap the locations in certain stops such as (l_2, l_1) at the beginning and (l_4, l_3) in the middle of the journey and still reach Denver because both G and O take into consideration the fact that the drone cannot travel more than k kilometers. Hence, each stop is within k kilometers of the next stop. G's stops go farther than O's stops, so G should have fewer or the same number of stops as O. Therefore, the greedy solution will be at least as good as the optimal solution. In other words, the greedy solution **is not worse than** the optimal solution.

   □

1.2 Problem 2 [30 pts]

Problem 2. You're organizing a university event spanning multiple days with student volunteers scheduled for various shifts. Each shift occurs during a specific time interval, and some shifts may overlap.

Your task is to select a **supervising committee** (a subset of students) so that:

- Each shift not in the committee is *partially overlapped* by at least one committee member's shift.

The goal is to form the **smallest possible committee** that ensures every shift is observed.

Problem:

Design an efficient algorithm that, given n shifts, finds the smallest supervising committee such that each shift is covered by at least one overlapping committee member.

Example:

If the shifts are:

- Shift 1: 4 PM – 8 PM
- Shift 2: 6 PM – 10 PM
- Shift 3: 9 PM – 11 PM

The smallest committee would consist of **just the second shift**, as it overlaps with both the first and third shifts.

Rubric	**Points**
Explain the key idea behind the algorithm (Greedy approach)	8 Points
Correctly describe and justify the approach	6 Points
Efficiently implement the algorithm with the correct time complexity	16 Points
Total	**30 Points**

Answer. **Key Idea/Greedy Approach**: We want to fill up a committee C with as few members as possible while meeting the conditions that all the shifts are covered. So we greedily choose the members that have the most overlapping shifts with non-committee members.

To do this, we choose the shift with the earliest finishing time. Then, we check with all the overlapping shifts and add the one with the latest finishing time S_l to the set of committee members (C). Then, we check which shifts overlap with shift S_l and delete them from the set. Repeat steps until the initial set of all intervals is empty.

Algorithm:

```
1 smallestCommittee(L) // L is the list of all shift intervals [S_i, E_i]
2    C = [] //contains set of committee members
3    sort(L) //by ending times
4    for i = 1 to n
5        I_1 = L[0] //pick earliest ending time
6        O = [] //List of overlaping shifts
7        for j = 1 to L.length
8           if L[j].S <= I_1.E and L[j].E >= I_1.S //if overlaps
9               O.append(L[j])
10       sort(O) //by end times in ascending order
11       I_2 = O[temp.length - 1]
12       C.append(I_2)
13       for k = 1 to L.length
14           if L[k].S <= I_2.E and L[k].E >= I_2.S //if overlaps
```

```
15                L.remove(L[k])
16    return C
```

Time Complexity: Sorting the set L takes $O(n \log n)$. The outer loop with run for $O(n)$ times since L has n elements. During each iteration, there could be up to $O(n)$ shifts that overlap with the chosen I_1 or I_2. So, each loop iteration runs $O(n)$ times. So, $T(n) = O(2n^2 + n \log n) = O(n^2)$. □

2 Examples Where Greedy Algorithms Fail

2.1 Problem 3 [15 pts]

Problem 3. Consider the Knapsack problem with the following inputs:

- A list of items: $[(v_1, w_1), \ldots, (v_n, w_n)]$, where v_i is the value and w_i is the weight of the i-th item.
- A weight limit W.

The goal is to select a subset $S \subseteq \{1, \ldots, n\}$ such that:

$$\sum_{i \in S} w_i \leq W$$

maximizes the total value:

$$V = \sum_{i \in S} v_i$$

.

Consider the following greedy algorithm for this problem:

1. Order the items by their value-per-weight ratio, v_i/w_i, in descending order.
2. Select items in this order, adding them to S as long as the total weight remains within the limit W.

Provide an example demonstrating that this greedy algorithm does not always yield the optimal solution. Include the example, show the steps of the greedy algorithm on it, and present a solution with a higher total value that adheres to the weight limit.

Rubric	**Points**
Clearly define an example with items and weight limit.	3 Points
Show step-by-step execution of the greedy algorithm on the example.	5 Point
Provide and justify an optimal solution with a higher value than the greedy solution.	4 Point
Explain why the greedy algorithm fails and how the optimal solution is found.	3 Point
Total	**15 Points**

Answer. **Example**:
Let there be a list of values and weights: $L = [(12, 2), (30, 10), (25, 5), (48, 12)]$ and let $W = 15$.
The greedy algorithm from above gives the solution below for list L and weight limit W.

1. Order the list by their value-per-weight ratio,
   $L_{v/w} = [6, 3, 5, 4] = [6, 5, 4, 3]$
2. $S = [(12, 2), (25, 5)]$
   The values add up to 37 and the weights add up to 7.

The solution that gives a higher value is $S' = [(48, 12), (12, 2)]$ because the values add up to $48 + 12 = 60$ but the weights add up to $12 + 2 = 14$. The total value is higher but the total weight is within the limit.

Explanation of why greedy algorithm fails:
The greedy algorithm fails because the ratio fails to acknowledge which of the pairs has a higher value. There will exist cases such as above where the pairs with a higher ratio have a lower value, but the greedy algorithm does not pick the pairs with the higher values because the ratios do not provide that information.
How the optimal solution is found:
To find the optimal solution, we can use the greedy algorithm provided with some backtracking.
The optimal solution is found by ordering the pairs by their value-per-weight ratio. Then we select the pair with the highest ratio, r_h. Then, we subtract the weight of that pair w_h from the weight limit $T_w = W - w_h$. If $T_w \neq 0$, then we look for the pair that has the highest value but the weight w_2 is still below $T_w + w_2 \leq W$. We repeat until $T_w > W$.

For instance, using the example from above,
1. Choose the highest ratio: $(12, 2)$
2. Remove the weight above from the weight limit: $T_w = 15 - 2 = 13$
3. Choose the highest value: $48, 12$ (Note: $12 < 13$ and $12 + 2 \leq 15$).
4. Remove weight from T_w: $T_w = 13 - 12 = 1$.
5. None of the pairs in the list have a weight of 1 or lower, so we are done with the solution set.

Solution: $S = [(12, 2), (48, 12)]$

□

3 Huffman Encoding

3.1 Problem 4 [15 pts]

Problem 4. Given the letter frequencies in the table below, perform the following tasks:

Letter	Count
E	3
H	2
L	2
O	1
R	1
T	1

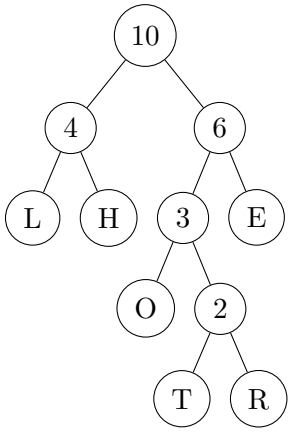
1. **Huffman Encoding:** Use the letter frequencies from the table to construct a Huffman tree. Based on this tree, generate variable-length binary codes for each letter, adhering to all conventions discussed in class. At each step, assign the subtree with lower frequency as the '0' subtree and the subtree with higher frequency as the '1' subtree. Break ties *lexicographically*: in the event of a tie, act as if the subtree containing the earliest letter in the alphabet has higher frequency. You may draw the tree by hand and upload a picture to your `.tex` file.
2. **Fixed-Length Encoding:** If fixed-length encoding were used, each letter would require the same number of bits. Determine the number of bits required and construct a table with fixed-length binary codes for each letter.
3. **Compression Ratio:** Calculate the compression ratio.
4. **Decoding:** Using the Huffman encoding generated in task (1), decipher the following encoded message:

   011100001001010011110111 1

Rubric	Points
Correctly construct the Huffman tree and generate the binary codes.	4 Points
Determine the correct number of bits and generate the codes.	4 Points
Accurately calculate the compression ratio.	3 Points
Correctly decode the encoded message.	4 Points
Total	**15 Points**

Answer.

1. Tree:



Variable Length Binary Codes for Each Letter:
Deriving the binary codes from the tree above,

Letter	**Binary Codes**
E	11
H	01
L	00
O	100
R	1011
T	1010

Variable Length Bit Calculation: $VL = 3 \cdot 2 + 2 \cdot 2 + 2 \cdot 2 + 1 \cdot 3 + 1 \cdot 4 + 1 \cdot 4 = 6 + 4 + 4 + 3 + 4 + 4 = 25$

2. Fixed-Length Encoding:
Since we are encoding 6 characters, we need **3** bits (allows max of 8 characters to be encoded). The possible binary codes are: 000, 001, 010, 011, 100, 101, 110

Fixed Length Binary Codes for each letter:

Letter	**Binary Codes**
E	000
H	101
L	010
O	100
R	011
T	001

Fixed Length Bit Calculation: $FL = 3 \cdot 3 + 2 \cdot 3 + 2 \cdot 3 + 1 \cdot 3 + 1 \cdot 3 + 1 \cdot 3 = 9 + 6 + 6 + 3 + 3 + 3 = 30$

3. Compression Ratio:
Using the FL and VL from above,

$$\begin{aligned} CR &= \frac{FL - VL}{FL} \cdot 100 \\ &= \frac{30 - 25}{30} \cdot 100 \\ &= \frac{5}{30} \cdot 100 \\ &= \frac{1}{6} \cdot 100 \\ &= \frac{100}{6} \\ &= 16.6667 \end{aligned}$$

4. Decoding:
01110000100101001111101111
01 11 00 00 100 1010 01 11 1011 11
H E L L O T H E R E

□